TECHNICAL DOCUMENTATION

Schema Drift Detector

AI-Powered Database Schema Monitoring & Drift Analysis

Detect. Classify. Recommend.

Frontend	Backend	AI Layer	Databases
React + Vite + Tailwind CSS	**FastAPI + SQLAlchemy**	**LangChain + OpenRouter**	**PostgreSQL SQLite**

Backend Deployed on **Render** ▲ Frontend Deployed on **Vercel**

1. Introduction

Every production system eventually runs into the same quiet problem: someone alters a database column, renames a table, or drops an index — and nothing breaks immediately. The pipeline keeps running. The APIs keep responding. Then, days later, something downstream quietly produces wrong numbers, or a nightly job fails at 2 AM with a cryptic type mismatch error.

Schema Drift Detector was built to catch those changes before they cause damage. It connects to your PostgreSQL or SQLite database, takes a structural snapshot, and then compares it against any future snapshot using a deterministic diff engine. More importantly, it doesn't just tell you what changed — it asks an AI to tell you what those changes mean.

The LLM layer (powered by LangChain and OpenRouter) classifies each change as Breaking, Additive, or Potentially Breaking, assigns a risk level, describes the downstream impact across ETL pipelines, REST APIs, analytics dashboards, and ML feature stores, and generates concrete mitigation steps. Everything is stored as a timestamped report, and a visual dashboard lets you track risk trends over time.

What makes this different

A lot of schema monitoring tools stop at detection. They tell you a column was dropped. Schema Drift Detector goes further — it applies AI reasoning to translate raw structural changes into business and engineering language that your whole team can act on.

Key Capabilities at a Glance

Capability	What It Does
Schema Snapshot	Captures the full structural definition of any connected database on demand
Drift Detection	Deterministic diff engine compares snapshots and enumerates every change with full detail
AI Classification	LLM labels each change type and assigns Low / Medium / High risk with a written justification
Impact Analysis	Explains effects on ETL pipelines, REST APIs, analytics dashboards, and ML feature stores
Mitigation Recommendations	Generates step-by-step remediation guidance for each detected change
Report History	Stores all drift reports with AI executive summaries, building a full audit trail over time
Risk Dashboard	Recharts-powered visual distribution of risk levels, updated after every scan

2. System Architecture

The application follows a clean three-tier architecture. The React SPA handles everything the user sees. The FastAPI service handles all business logic — connecting to databases, running diffs, orchestrating the AI agent. SQLAlchemy acts as the data layer both for the internal application database (where snapshots and reports are stored) and for reflecting the schemas of monitored databases.

2.1 The Three Layers

Layer	Technology	Responsibility
Presentation	React + Vite + Tailwind CSS + Recharts	Single-page app: forms, dashboard charts, report viewer, snapshot browser
Application	FastAPI (Python 3.10+)	REST API, request validation, scan orchestration, AI agent coordination
AI Agent	LangChain + OpenRouter (GPT-3.5-turbo)	Prompt construction, LLM call, structured JSON response parsing and validation
Data Access	SQLAlchemy ORM + Reflection	Internal data persistence; runtime schema inspection of monitored databases
Storage	PostgreSQL / SQLite	Snapshot history, drift reports, connection configurations

2.2 How a Scan Works — End to End

Understanding the data flow helps when something goes wrong or when you want to extend the system. Here is what happens from the moment you click Run AI Scan to the moment you see results on screen.

1. The frontend sends a POST /scans request to the FastAPI backend, passing the connection ID.
2. The Schema Extractor service opens a SQLAlchemy engine pointed at your target database and uses reflection to enumerate every table, column, data type, constraint, index, and foreign key relationship.
3. The Snapshot Service serializes the schema as a JSON document and persists it with a timestamp. If this is the first scan on a connection, it saves the baseline and exits — there's nothing to compare yet.

4. The Diff Engine performs a deterministic comparison of the new snapshot against the previous one, producing a structured list of change records categorized by type (column added, table dropped, type changed, and so on).
5. The Drift Analyzer Agent (LangChain) constructs a prompt from the change list and sends it to OpenRouter. The LLM returns a structured JSON response with classification, risk level, impact narrative, and mitigation steps for each change, plus an executive summary.
6. The full result is persisted as a Drift Report and returned to the frontend, which renders it on the Drift Reports page and updates the Dashboard charts.

3. Project Structure

The repository is organized into two top-level directories — backend and frontend — plus a docker-compose.yml at the root for local containerized development. Each side is self-contained and can be developed, tested, and deployed independently.

```
Schema_Drift_Detector/
├── backend/
│   ├── app/
│   │   ├── agents/          # LangChain drift analyzer agent
│   │   ├── api/             # FastAPI route handlers
│   │   ├── models/          # SQLAlchemy ORM models
│   │   └── services/        # Schema extractor, diff engine, snapshot service
│   ├── .env.example
│   └── requirements.txt
├── frontend/
│   ├── src/
│   │   ├── components/      # Sidebar, badges, shared UI components
│   │   └── pages/           # Dashboard, ConnectDB, Scan, Reports, Snapshots
│   └── .env
└── docker-compose.yml
```

3.1 Backend Modules

The backend is organized around the principle of single responsibility. Each module does one thing well, which makes testing and extending the system straightforward.

Module	What It Does
agents/drift_analyzer.py	Builds the LangChain prompt from the change list, calls OpenRouter, parses and validates the JSON response. Retries with a simplified prompt on parse failure.
api/connections.py	CRUD endpoints for database connection configurations. Tests connectivity before saving.
api/snapshots.py	Endpoints for creating and retrieving schema snapshots.
api/scans.py	The main POST /scans endpoint that orchestrates the entire snapshot-diff-AI pipeline.
api/reports.py	Read-only endpoints for listing and retrieving drift reports with full AI analysis.
models/connection.py	ORM model storing database connection credentials and metadata.
models/snapshot.py	ORM model storing serialized schema snapshots as JSON blobs with timestamps.

Module	What It Does
models/report.py	ORM model storing drift reports including all AI-generated fields.
services/schema_extractor.py	Uses SQLAlchemy introspection to reflect tables, columns, types, constraints, and indexes.
services/diff_engine.py	Deterministic diffing logic that produces typed ChangeRecord objects for each structural difference.
services/snapshot_service.py	Coordinates snapshot creation, retrieval, and comparison selection logic.

3.2 Frontend Pages

Page	Purpose
Dashboard	Risk distribution charts (Recharts pie and bar), summary stats, and a feed of recent scan activity.
Connect DB	Form to add PostgreSQL or SQLite connections. Lists and manages saved connections.
Run Scan	Select a connection and trigger an AI scan. Shows live status and change counts.
Drift Reports	Paginated history of all drift reports with per-change classification badges, risk levels, impact explanations, mitigation steps, and AI executive summary.
Snapshots	Browse and inspect all stored schema snapshots, searchable by database and timestamp.

4. Installation & Setup

You can run this project in two ways: directly on your machine (recommended during development) or inside Docker containers (recommended for anything resembling production). Both approaches are covered below.

4.1 Prerequisites

Before you start, make sure you have the following installed and accessible on your machine:

- Python 3.10 or newer
- Node.js 18 or newer, with npm
- PostgreSQL 14+ if you want to use it as the internal app database (SQLite works fine locally)
- Docker and Docker Compose if you're going the container route
- An OpenRouter account with an API key — sign up at openrouter.ai

4.2 Getting the Code

```
git clone https://github.com/yourusername/schema-drift-detector.git
cd schema-drift-detector
```

4.3 Running the Backend

The backend is a standard Python project. Navigate into the backend directory, install dependencies, configure your environment, and start the server.

```
cd backend
pip install -r requirements.txt

# Copy the env template and fill in your values
cp .env.example .env

# Start the development server
uvicorn app.main:app --reload
```

Once running, the API lives at http://localhost:8000. Swagger UI (interactive API docs) is at http://localhost:8000/docs — it's a great place to test endpoints directly without touching the frontend.

4.4 Running the Frontend

```
cd frontend
npm install
npm run dev
```

The React app will be available at http://localhost:5173. Make sure VITE_API_BASE_URL in frontend/.env points to your running backend.

4.5 Running with Docker

If you'd rather not manage the two processes separately, Docker Compose spins up everything together — frontend, backend, and an internal PostgreSQL instance.

```
# Build and start all services
docker-compose up --build

# Run in background
docker-compose up --build -d

# Tail backend logs
docker-compose logs -f backend

# Stop everything
docker-compose down
```

5. Environment Variables

All sensitive configuration is handled through environment variables. The backend expects a .env file in the backend/ directory. The frontend expects a .env file in the frontend/ directory. Never commit either of these files to version control — the .env.example file is provided as a safe template to commit instead.

5.1 Backend (.env)

Variable	Example	Description
OPENROUTER_API_KEY	sk-or-v1-...	Your OpenRouter API key. Required for all AI features.
OPENROUTER_BASE_URL	https://openrouter.ai/api/v1	OpenRouter base URL. Leave this as-is unless OpenRouter changes it.
LLM_MODEL	openai/gpt-3.5-turbo	The model identifier passed to OpenRouter. Swap this to use a different model.
DATABASE_URL	postgresql://user:pw@host/db	Connection string for the internal app database where snapshots and reports are stored.
SECRET_KEY	a-long-random-string	Used for internal token signing. Generate a random 32+ character string.
CORS_ORIGINS	https://your-app.vercel.app	Comma-separated list of allowed frontend origins. Critical for production security.

5.2 Frontend (.env)

Variable	Example	Description
VITE_API_BASE_URL	https://your-backend.onrender.com	The full base URL of the FastAPI backend. No trailing slash.

6. Deployment

The project is designed to deploy cleanly on the two most popular modern hosting platforms for this kind of stack: Render for the FastAPI backend and Vercel for the React frontend. Both platforms offer free tiers that are sufficient for development and small production workloads.

6.1 Backend — Render

Render is well-suited for Python web services. It supports native Python runtimes without requiring you to write a Dockerfile (though that option exists too). Here is how to get the backend running on Render.

Step-by-step deployment on Render

7. Go to render.com and create a new Web Service.
8. Connect your GitHub repository. Render will detect the backend directory automatically, or you can specify it manually.
9. Set the Root Directory to backend.
10. Set the Build Command to:

```
pip install -r requirements.txt
```

11. Set the Start Command to:

```
uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

12. Under the Environment tab, add all your backend environment variables (OPENROUTER_API_KEY, DATABASE_URL, SECRET_KEY, CORS_ORIGINS, etc.). Set CORS_ORIGINS to your Vercel frontend URL.
13. Click Deploy. Render will build and start your service. The public URL (e.g. https://schema-drift-backend.onrender.com) is what you'll use for VITE_API_BASE_URL in Vercel.

Using a Render PostgreSQL database

Render offers managed PostgreSQL databases. Create one from the dashboard and copy the Internal Database URL (use the external URL if your app is outside Render's network). Set this as your DATABASE_URL environment variable on the Web Service.

Cold Starts on the Free Tier

Render's free tier spins down services after 15 minutes of inactivity. The first request after a sleep may take 30–60 seconds to respond while the instance wakes up. Upgrade to a paid plan or use an uptime monitor (like UptimeRobot) to keep it warm.

6.2 Frontend — Vercel

Vercel is designed for exactly this kind of React + Vite project. Deployment is nearly automatic once the repository is connected.

Step-by-step deployment on Vercel

14. Go to vercel.com and import your GitHub repository.
15. Set the Root Directory to frontend.
16. Vercel auto-detects Vite. The Framework Preset will be set to Vite automatically. Build Command is npm run build and Output Directory is dist.
17. Under Environment Variables, add VITE_API_BASE_URL and set it to your Render backend URL.
18. Click Deploy. Vercel builds the project and gives you a URL like https://schema-drift-detector.vercel.app. Every push to your main branch triggers an automatic redeploy.

Handling client-side routing

Because Schema Drift Detector is a single-page app, Vercel needs to know that all routes should be served from index.html. Create a vercel.json file in the frontend/ directory:

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

6.3 The Full Deployed Architecture

Component	Platform	URL Pattern	Notes
React Frontend	Vercel	https://your-app.vercel.app	Auto-deploys on every push to main
FastAPI Backend	Render	https://your-backend.onrender.com	Free tier sleeps after 15 min idle
PostgreSQL Database	Render (managed)	Internal to Render network	Use external URL for external connections

6.4 Production Checklist

Before going live, work through this list:

- CORS_ORIGINS is set to your exact Vercel domain — not a wildcard
- OPENROUTER_API_KEY is stored securely as an environment variable, never hardcoded
- DATABASE_URL points to a managed PostgreSQL instance, not a local SQLite file
- The Render service is on at least the Starter paid plan if uptime matters (eliminates cold starts)
- vercel.json is present in frontend/ to handle client-side routing correctly
- VITE_API_BASE_URL matches the exact Render backend URL with no trailing slash

7. Using the Application

Once everything is running — locally or deployed — using Schema Drift Detector follows a simple, repeatable workflow.

7.1 Connecting a Database

Start by navigating to Connect DB in the sidebar. Select PostgreSQL or SQLite, fill in the connection details, and click Connect. The backend tests the connection before saving it, so you'll know immediately if something is wrong with the credentials or host.

For PostgreSQL you'll need: host, port, database name, username, and password. For SQLite, you just need the absolute path to the .db file on the server running the backend.

7.2 Taking Your First Snapshot

Go to Run Scan, select your connection from the dropdown, and click Run AI Scan. On the first run, the system takes a baseline snapshot and tells you so — there's no previous state to compare against yet, so no drift report is generated. That's expected behavior.

Now go make some schema changes in your database: add a column, change a type, drop a table. Then come back and run the scan again. This time, the Diff Engine will find the changes and the AI agent will analyze them.

7.3 Reading a Drift Report

Drift Reports is where the analysis lives. Each report shows the scan timestamp, the connection name, and a list of every change detected. For each change you'll see:

- A classification badge — Breaking, Additive, or Potentially Breaking
- A risk level badge — High, Medium, or Low — with color coding
- An impact explanation written in plain language describing what could break and why
- Mitigation steps: concrete actions to take to address the change safely

At the top of each report is an AI-generated executive summary — a paragraph that synthesizes all the changes into a coherent assessment of the overall drift event, useful for sharing with non-technical stakeholders.

7.4 Understanding the Risk Levels

Classification	Risk	Typical Examples
Breaking	High	Column dropped, table dropped, data type changed incompatibly, NOT NULL constraint added to populated column
Potentially Breaking	Medium	Index removed, constraint changed, column renamed, default value removed
Additive	Low	New table added, nullable column added, new index added, default value added to optional column

8. API Reference

The FastAPI backend exposes a fully documented REST API. When running locally, Swagger UI is available at http://localhost:8000/docs — you can test any endpoint directly from the browser without writing any code.

8.1 Connections

Method	Endpoint	Description
POST	/connections	Create and test a new database connection. Returns the saved connection record.
GET	/connections	List all saved connections.
GET	/connections/{id}	Get a specific connection by ID.
DELETE	/connections/{id}	Delete a connection record. Does not affect the target database.

8.2 Snapshots

Method	Endpoint	Description
POST	/snapshots	Manually trigger a snapshot for a given connection_id.
GET	/snapshots	List all snapshots. Accepts an optional connection_id query parameter to filter.
GET	/snapshots/{id}	Retrieve a specific snapshot including the full schema JSON.

8.3 Scans

Method	Endpoint	Description
POST	/scans	Run a full scan: snapshot + diff + LLM analysis. Returns the drift report.

Request body for POST /scans:

```
{
  "connection_id": "uuid-of-your-connection",
  "compare_to": "uuid-of-baseline-snapshot"   // optional; defaults to latest
}
```

8.4 Reports

Method	Endpoint	Description
GET	/reports	List all drift reports with summary metadata (timestamp, connection, change count, risk distribution).
GET	/reports/{id}	Get a full report including all change records and the complete AI analysis.

9. The AI Drift Analyzer

The AI layer is what makes Schema Drift Detector useful rather than just technically correct. Raw diff output tells you that a column was dropped. The AI tells you which downstream systems that probably broke, why it matters, and what to do about it.

9.1 How the Agent Works

The Drift Analyzer agent is built with LangChain and sends requests to OpenRouter, which acts as a unified gateway to multiple LLM providers. For each scan, the agent:

- Receives the structured list of ChangeRecord objects from the Diff Engine
- Builds a detailed system prompt establishing the agent's role as a database schema risk analyst
- Formats the change list as structured JSON within the user prompt
- Instructs the model to return a JSON array with one object per change, plus an executive summary
- Parses the response, validates field values against allowed enums, and falls back to safe defaults if validation fails
- On JSON parse failure, retries once with a simplified prompt before surfacing an error

9.2 What the LLM Returns

For each detected change, the model returns:

```
{
  "change_type": "COLUMN_DROPPED",
  "classification": "Breaking",            // Breaking | Additive |
Potentially Breaking
  "risk_level": "High",                    // High | Medium | Low
  "impact_explanation": "Removing the user_email column will break...",
  "mitigation_steps": "1. Audit all queries referencing user_email..."
}
```

Plus an executive_summary string at the top level, summarizing the entire drift event in two to four sentences.

9.3 Changing the Model

The model is controlled entirely by the LLM_MODEL environment variable. Any model available on OpenRouter can be used. Here are some well-tested options:

Model ID	Provider	Best For
openai/gpt-3.5-turbo	OpenAI	Default choice. Fast, cheap, good structured output.
openai/gpt-4o	OpenAI	More nuanced analysis. Recommended for large or complex schemas.
anthropic/claude-3-haiku	Anthropic	Fast with strong instruction following for JSON responses.
meta-llama/llama-3-8b-instruct	Meta (via OpenRouter)	Open-source option. Lower cost, slightly less consistent JSON.

10. Schema Extractor & Diff Engine

10.1 What the Extractor Captures

The Schema Extractor uses SQLAlchemy's runtime reflection API, which means it works against any database engine SQLAlchemy supports — without requiring you to write any database-specific code. For each connected database, the extractor captures:

- All table names and table-level comments
- Every column: name, data type, nullable flag, default value
- Primary key constraints and their constituent columns
- Foreign key relationships: referencing table, referencing column, referenced table, referenced column
- Unique constraints
- All indexes: name, columns covered, whether unique
- Check constraints (PostgreSQL only)

The output is serialized as a JSON document and stored as a Snapshot record. Each snapshot is immutable — once taken, it represents a point-in-time truth about your schema.

10.2 How the Diff Engine Works

The Diff Engine is deliberately deterministic. Given the same two snapshots, it always produces the same output in the same order. This matters for reliable AI analysis — the LLM is less likely to hallucinate if the input is consistently structured.

The engine compares tables, then columns within matched tables, then constraints and indexes. Here are all the change categories it can detect:

Change Category	What Triggered It
TABLE_ADDED	A table exists in the new snapshot that wasn't in the baseline
TABLE_DROPPED	A table in the baseline is absent from the new snapshot
COLUMN_ADDED	A column exists in a table in the new snapshot that wasn't there before
COLUMN_DROPPED	A column present in the baseline is missing from the new snapshot
COLUMN_TYPE_CHANGED	The data type of an existing column has been modified
COLUMN_NULLABLE_CHANGED	The nullable flag on a column has been toggled

Change Category	What Triggered It
COLUMN_DEFAULT_CHANGED	A default value has been added, removed, or changed
INDEX_ADDED	A new index has been created on one or more columns
INDEX_DROPPED	An existing index has been dropped
CONSTRAINT_CHANGED	A primary key, foreign key, or unique constraint has been modified

11. Troubleshooting

Most issues fall into one of three categories: connectivity (the frontend can't reach the backend), configuration (a missing or wrong environment variable), or LLM behavior (the AI returns unexpected output). Here's a rundown of the most common problems and how to fix them.

Problem	Likely Cause & Fix
API calls fail in the browser (CORS error)	The frontend origin is not in CORS_ORIGINS on the backend. Add your exact Vercel URL to CORS_ORIGINS and redeploy the backend on Render.
Render backend is very slow on first request	The free tier spins down after 15 minutes of inactivity. Either upgrade the plan, use UptimeRobot to ping the service regularly, or warn users to expect a cold-start delay.
LLM returns empty or invalid JSON	Check that OPENROUTER_API_KEY is valid and has credits. Try switching LLM_MODEL to gpt-4o for more reliable structured output. The agent retries once automatically.
No changes detected after modifying the schema	Make sure you ran a second scan after making changes. Confirm the right connection is selected. Check that the schema changes were committed (not inside an uncommitted transaction).
PostgreSQL connection refused	Verify DATABASE_URL credentials, host, and port. If using Render's database, use the External URL when connecting from outside Render's network (e.g. from your local machine).
Frontend shows blank page on Vercel	Check that VITE_API_BASE_URL is set correctly in Vercel's environment variables. Also confirm vercel.json exists in the frontend/ directory for SPA routing.
Docker build fails on requirements.txt	Ensure the base image uses Python 3.10+. Run pip install --upgrade pip before installing requirements to rule out pip version issues.

12. Contributing

Contributions are welcome — whether that's a bug fix, a new change category in the diff engine, support for an additional database type, or improvements to the AI prompting strategy. Here's how the project is organized for contributors.

12.1 Branch Strategy

- main — stable, production-ready releases only
- develop — integration branch for feature work; all PRs target this branch
- feature/your-feature-name — short-lived branches off develop for new features
- fix/issue-description — hotfix branches off main for urgent production bugs

12.2 Code Style

The Python codebase follows PEP 8. Use Black for formatting and isort for import ordering — both are in the dev dependencies. The JavaScript/React side uses ESLint with the project's config and Prettier for formatting. For commit messages, use Conventional Commits format (feat:, fix:, docs:, refactor:, chore:, etc.).

12.3 Pull Request Expectations

Before opening a PR, please make sure:

- All existing tests pass (pytest for backend, vitest for frontend)
- New features include unit tests for any new service methods or API endpoints
- If you've added environment variables, they're documented in .env.example and in Section 5 of this document
- The PR description explains the change clearly, the reason for it, and how you tested it

13. License